

# Monday reflection: design process mapping

---

Written after actually challenging the process map, not before. This closes the gap the audit page itself flagged: "that back-and-forth happened live, but wasn't captured in writing."

## What design owns vs. what engineering owns at Ravn

Design owns what something means and how it communicates that meaning: deciding verification needs 4 real states instead of a yes/no, deciding trust has to be shape-plus-color so no one is excluded.

Engineering owns turning that meaning into working, correct code: the actual data type, the actual keyboard behavior, the actual component wired to the actual token.

## Engineering's contribution across the lifecycle

The Architect phase's "nullable rating" wasn't just a technical footnote, it shaped a design decision. A brand-new provider has zero reviews, so the rating field can't default to a number; it has to be allowed to be empty. That engineering constraint is why a "No rating yet" state exists at all in the real card states. Design had to design around a limit engineering identified first, not the other way around.

## One design decision, reframed as a system decision

"Verification becomes 4 states, not a boolean" looks like a visual choice, just how many badges to draw. It mostly isn't. Verification in real life is a multi-step process (submitted, under review, background-checked, rejected), and "currently being reviewed" cannot live in a field that only allows true or false. That part of the decision is fully forced by the data; design had no real discretion there.

But the actual design decision was hiding one layer down: the data needing 4 states doesn't force the requester-facing UI to render all 4 distinct states. A requester arguably only needs "not verified yet, don't fully trust this," so pending and rejected could collapse into one neutral signal from their side, while admins and providers, who act on the difference between "rejected" and "still pending," need to see all 4. Which states get their own visual treatment, and for which role, is a real design decision, and it's the same three-role split from Discover: what a requester needs to know and what an admin needs to know aren't the same thing, even though both are reading the same underlying field.

## Two documented pushbacks

**1. Handoff.** The real risk this week wasn't token drift, it was meaning loss: the verified mark got rebuilt from scratch as a plain circle with no accessible label. The tokens were even close to correct, but the meaning was gone. A handoff step for Vello specifically needs to say: "trust-carrying elements (VerifiedMark, Badge, Rating) hand off as real components, not just their token values," because for Vello, unlike a generic app, the trust signal *is* the product, not decoration on top of it.

**2. Validate.** First draft of this pushback claimed the whole Validate phase was generic ("just standard WCAG checklist stuff"). That didn't survive checking it against the real artifact text, which actually includes "whether 'unverified' is still honestly visible instead of quietly hidden." That's not generic at all, it's checking the one failure mode Vello's whole trust bet depends on not happening.

The real, narrower critique: Validate lists all three checks (contrast ratio, tap target size, and unverified-visibility) side by side, with no signal that two are routine and one is load-bearing. Someone skimming this phase, or an AI given it as a spec, has no reason to weight "is unverified honestly visible" any higher than "is contrast 4.5:1." If anything, the Vello-specific check is the one most likely to get silently deprioritized, because it reads like it belongs in the same bucket as the boilerplate items next to it.